CIRCULAR DOUBLY LINKED LIST

1.Insertion at beginning

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};
struct Node* insertBegin(struct Node *head, int data)
{
    struct Node *newnode = (struct Node*)malloc(sizeof(struct Node
        ));
    newnode->data = data;
    if(head == NULL)
    {
        newnode->next = newnode;
        newnode->prev = newnode;
        head = newnode;
        return head;
    }
    struct Node *last = head->prev;
    newnode->next = head;
    newnode->prev = last;
    last->next = newnode;
    head->prev = newnode;
    head = newnode;
```

```
30 40 50

=== Code Execution Successful ===
```

main.c

```c
    newnode->next = head;
    newnode->prev = last;
    last->next = newnode;
    head->prev = newnode;
    head = newnode;
    return head;
}
void display(struct Node *head)
{
    if(head == NULL) return;
    struct Node *temp = head;
    do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    while(temp != head);
}
int main()
{
    struct Node *head = NULL;
    head = insertBegin(head, 50);
    head = insertBegin(head, 40);
    head = insertBegin(head, 30);
    display(head);
}
```

Share Run Output

```
30 40 50

=== Code Execution Successful ===
```

2.Insertion at given position

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};
struct Node* insertPos(struct Node *head, int data, int pos)
{
    struct Node *newnode = (struct Node*)malloc(sizeof(struct Node
        ));
    newnode->data = data;
    // Case 1: Empty list
    if(head == NULL)
    {
        newnode->next = newnode;
        newnode->prev = newnode;
        return newnode;
    }
    if(pos == 1)
    {
        struct Node *last = head->prev;
        newnode->next = head;
        newnode->prev = last;
        last->next = newnode;
```

Output

```
5 10 12 15

=== Code Execution Successful ===
```

main.c

```c
        head = newnode;
        return head;
    }
    struct Node *temp = head;
    for(int i = 1; i < pos-1; i++)
        temp = temp->next;
    newnode->next = temp->next;
    newnode->prev = temp;
    temp->next->prev = newnode;
    temp->next = newnode;
    return head;
}
void display(struct Node *head)
{
    if(head == NULL)
    {
        printf("List is empty");
        return;
    }
    struct Node *temp = head;
    do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
```

Output

```
5 10 12 15

=== Code Execution Successful ===
```

```c
}
void display(struct Node *head)
{
    if(head == NULL)
    {
        printf("List is empty");
        return;
    }
    struct Node *temp = head;
    do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    while(temp != head);
}
int main()
{
    struct Node *head = NULL;
    head = insertPos(head, 5, 1);
    head = insertPos(head, 10, 2);
    head = insertPos(head, 15, 3);
    head = insertPos(head, 12, 3);
    display(head);
    return 0;
}
```

```
5 10 12 15

=== Code Execution Successful ===
```

3.Insertion at end

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};
struct Node* insertEnd(struct Node *head, int data)
{
    struct Node *newnode = malloc(sizeof(struct Node));
    newnode->data = data;
    if(head == NULL)
    {
        newnode->next = newnode;
        newnode->prev = newnode;
        return newnode;
    }
    struct Node *last = head->prev;
    last->next = newnode;
    newnode->prev = last;
    newnode->next = head;
    head->prev = newnode;
    return head;
}
void display(struct Node *head)
```

```
11 22 33

=== Code Execution Successful ===
```

main.c

```c
    }
    struct Node *last = head->prev;
    last->next = newnode;
    newnode->prev = last;
    newnode->next = head;
    head->prev = newnode;
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
   do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    while(temp != head);
}
int main()
{
    struct Node *head = NULL;
    head = insertEnd(head, 11);
    head = insertEnd(head, 22);
    head = insertEnd(head, 33);
    display(head);
}
```

Output

```
11 22 33

=== Code Execution Successful ===
```

4.deletion at beginning

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};
struct Node* deleteBegin(struct Node *head)
{
    if(head == NULL)
        return NULL;
    if(head->next == head)
    {
        free(head);
        return NULL;
    }
    struct Node *last = head->prev;
    struct Node *temp = head;
    head = head->next;
    head->prev = last;
    last->next = head;
    free(temp);
    return head;
}
void display(struct Node *head)
```

Output

```
200 300

=== Code Execution Successful ===
```

main.c

```c
{
    struct Node *temp = head;
    do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    while(temp != head);
}
int main()
{
    struct Node *head = NULL;
    struct Node *n1 = malloc(sizeof(struct Node));
    struct Node *n2 = malloc(sizeof(struct Node));
    struct Node *n3 = malloc(sizeof(struct Node));
    n1->data = 100;
    n2->data = 200;
    n3->data = 300;
    head = n1;
    n1->next = n2;
    n1->prev = n3;
    n2->next = n3;
    n2->prev = n1;
    n3->next = n1;
    n3->prev = n2;
```

Output

```
200 300

=== Code Execution Successful ===
```

5.deletion at given position

Programiz C Online Compiler

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};
struct Node* deletePos(struct Node *head, int pos)
{
    if(head == NULL)
    {
        printf("List is empty\n");
        return head;
    }
    struct Node *temp = head;
    // Delete first node
    if(pos == 1)
    {
        if(head->next == head)
        {
            free(head);
            return NULL;
        }
        struct Node *last = head->prev;
        head = head->next;
```

Output

```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successful ===
```

Programiz C Online Compiler

```c
        last->next = head;
        head->prev = last;
        free(temp);
        return head;
    }
    for(int i = 1; i < pos; i++)
        temp = temp->next;
    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
    free(temp);
    return head;
}
void display(struct Node *head)
{
    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
```

Output

```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successful ===
```

Programiz C Online Compiler

```c
    while(temp != head);
    printf("\n");
}
int main()
{
    struct Node *head = NULL;
    struct Node *n1, *n2, *n3;
    n1 = (struct Node*)malloc(sizeof(struct Node));
    n2 = (struct Node*)malloc(sizeof(struct Node));
    n3 = (struct Node*)malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->next = n2;
    n1->prev = n3;
    n2->next = n3;
    n2->prev = n1;
    n3->next = n1;
    n3->prev = n2;
    printf("Before deletion:\n");
    display(head);
    head = deletePos(head, 2);
    printf("After deletion:\n");
    display(head);
    return 0;
```

Output

```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successful ===
```

6.deletion at end

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};
struct Node* deleteEnd(struct Node *head)
{
    if(head == NULL)
    {
        printf("List is empty\n");
        return NULL;
    }
    // Case: Only one node
    if(head->next == head)
    {
        free(head);
        return NULL;
    }
    struct Node *last = head->prev;
    struct Node *secondLast = last->prev;
    secondLast->next = head;
    head->prev = secondLast;
    free(last);
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20


=== Code Execution Successful ===
```

main.c

```c
    return head;
}
void display(struct Node *head)
{
    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    while(temp != head);
    printf("\n");
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = (struct Node*)malloc(sizeof(struct Node));
    n2 = (struct Node*)malloc(sizeof(struct Node));
    n3 = (struct Node*)malloc(sizeof(struct Node));
```

Output


```
Before deletion:
10 20 30
After deletion:
10 20


=== Code Execution Successful ===
```

Programiz
C Online Compiler

main.c

Share | Run | Output | Clear

```c
    printf("\n");
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = (struct Node*)malloc(sizeof(struct Node));
    n2 = (struct Node*)malloc(sizeof(struct Node));
    n3 = (struct Node*)malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->next = n2;
    n1->prev = n3;
    n2->next = n3;
    n2->prev = n1;
    n3->next = n1;
    n3->prev = n2;
    printf("Before deletion:\n");
    display(head);
   head = deleteEnd(head);
    printf("After deletion:\n");
    display(head);
    return 0;
}
```

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```